School System

Desenvolvimento de Software e Sistemas Móveis (DSSMV)

Licenciatura em Engenharia de Telecomunicações e Informática

LETI/ISEP

2026/27

Paulo Baltarejo Sousa

pbs@isep.ipp.pt

Instituto Superior de Engenharia do Porto

P.PORTO

Disclaimer

Material and Slides

Some of the material/slides are adapted from various:

- Presentations found on the internet;
- Books;
- Web sites;
- ...

Outline

1. **Context/Problem**
2. **School Disk API**
3. **School Disk Client**
4. **School Disk Client: Technical Report**
   - Context/Problem
   - Analysis
   - Design
   - Implementation
   - Tests
   - Conclusion

1 Context/Problem

2 School Disk API

3 School Disk Client

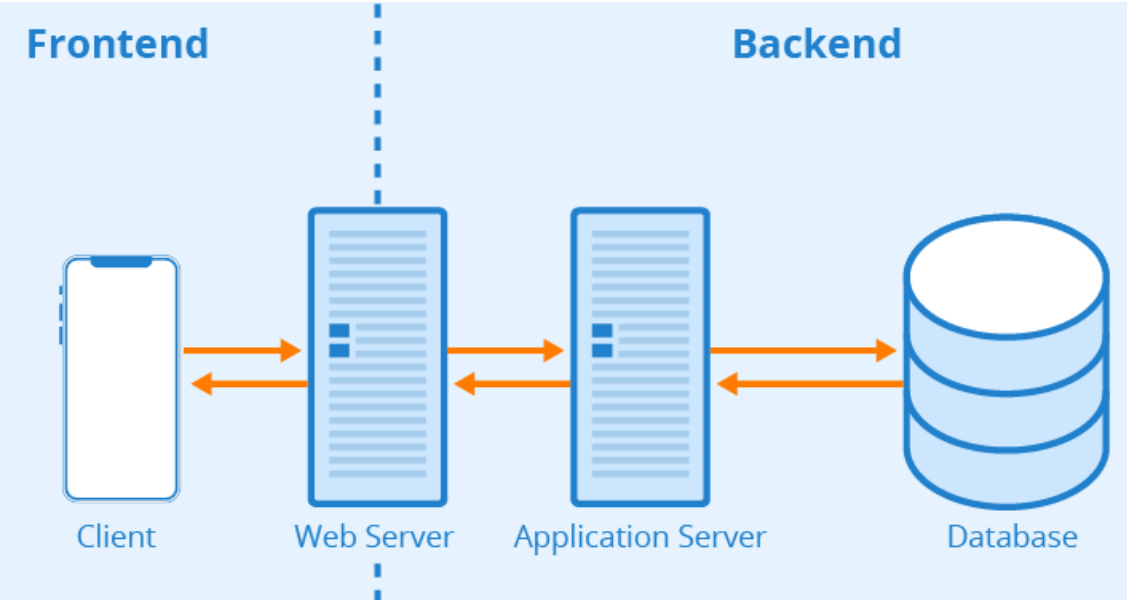
4 School Disk Client: Technical Report
- Context/Problem
- Analysis
- Design
- Implementation
- Tests
- Conclusion

Context/Problem

- School features:
  - School has students, instructors and offers subjects.
  - Instructors can teach many subjects.
  - The same subject can be taught by different instructors.
  - Student can attend to many subjects.
- School functionalities:
  - CRUD Students
  - CRUD Subjects
  - ...

School System

- Backend: `School Disk API` is a REST API web service.
  - Implements the business logical
  - It provides a set of functionalities to manage a school [1].
- Frontend: `School Disk Client` is a mobile App
  - Implements the user interface for the School system
  - Interacts with `School Disk API`



[1] Using `http://localhost:8585/swagger-ui.html`

1 Context/Problem

2 School Disk API

3 School Disk Client

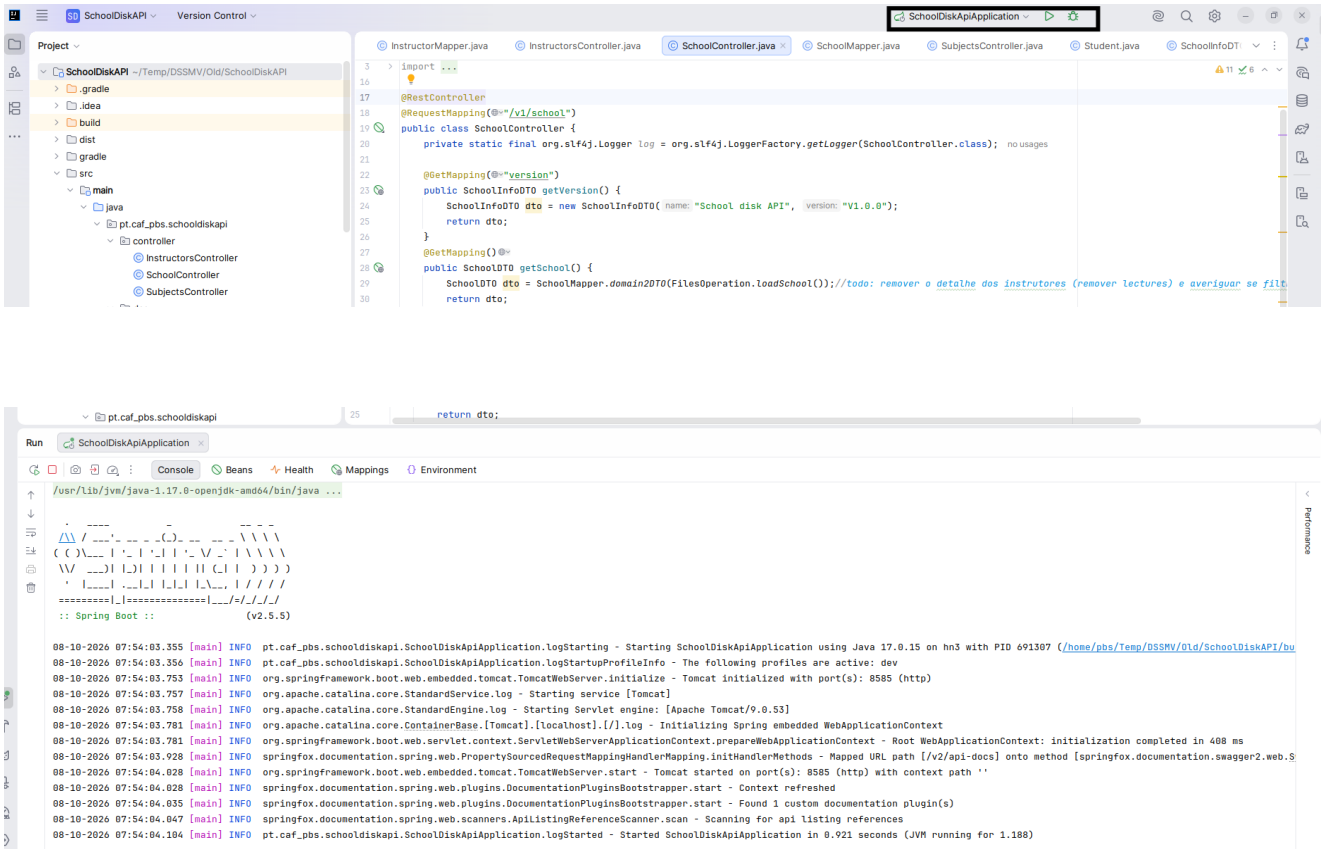
4 School Disk Client: Technical Report
- Context/Problem
- Analysis
- Design
- Implementation
- Tests
- Conclusion

Setting up

- To run it on your computer download it from moodle the zip named: `SchoolDiskAPI.zip`
- Unzip it.
- Install JAVA 11 in your PC. Provably it is already installed.

Boot: IntelliJ IDE

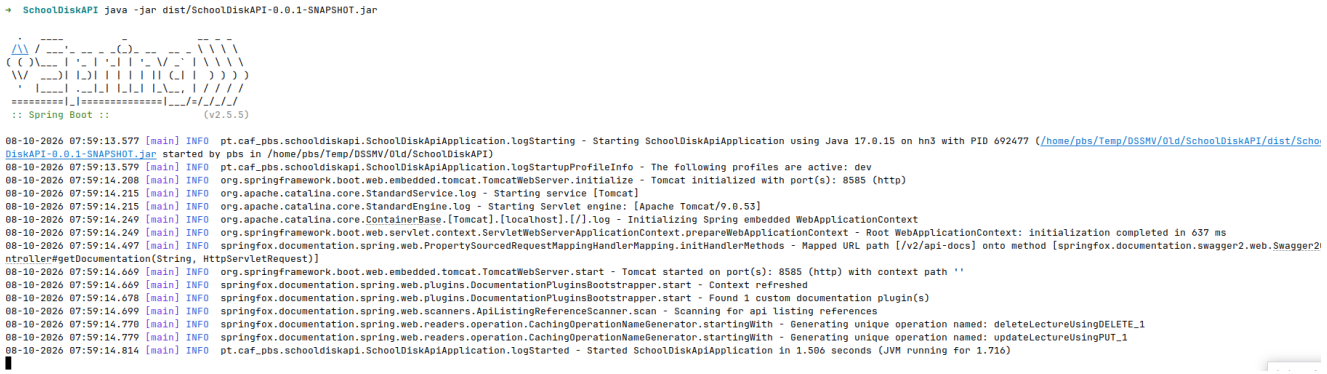
- In order to open and run this project from IntelliJ IDEA it is required the Spring Boot plugin [2]



[2] https://www.jetbrains.com/help/idea/spring-boot.html

Boot: Java

- In the folder were you extracted `SchoolDiskAPI.zip` run the following command:
  - `java -jar dist/SchoolDiskAPI-0.0.1-SNAPSHOT.jar`



- If it fails you can also try (PAY ATENTION to the JDK version number which may vary from one PC to other):
  - `"C:\Program Files\Amazon Corretto\jdk11.0.12_7\bin\java.exe" -jar dist/SchoolDiskAPI-0.0.1-SNAPSHOT.jar`

Boot: Gradle

- In the folder were you extracted `SchoolDiskAPI.zip` run the following command:
  - `.\gradlew.bat bootRun` (windows OS)
  - `.\gradlew bootRun` (Linux OS)



Boot: Docker

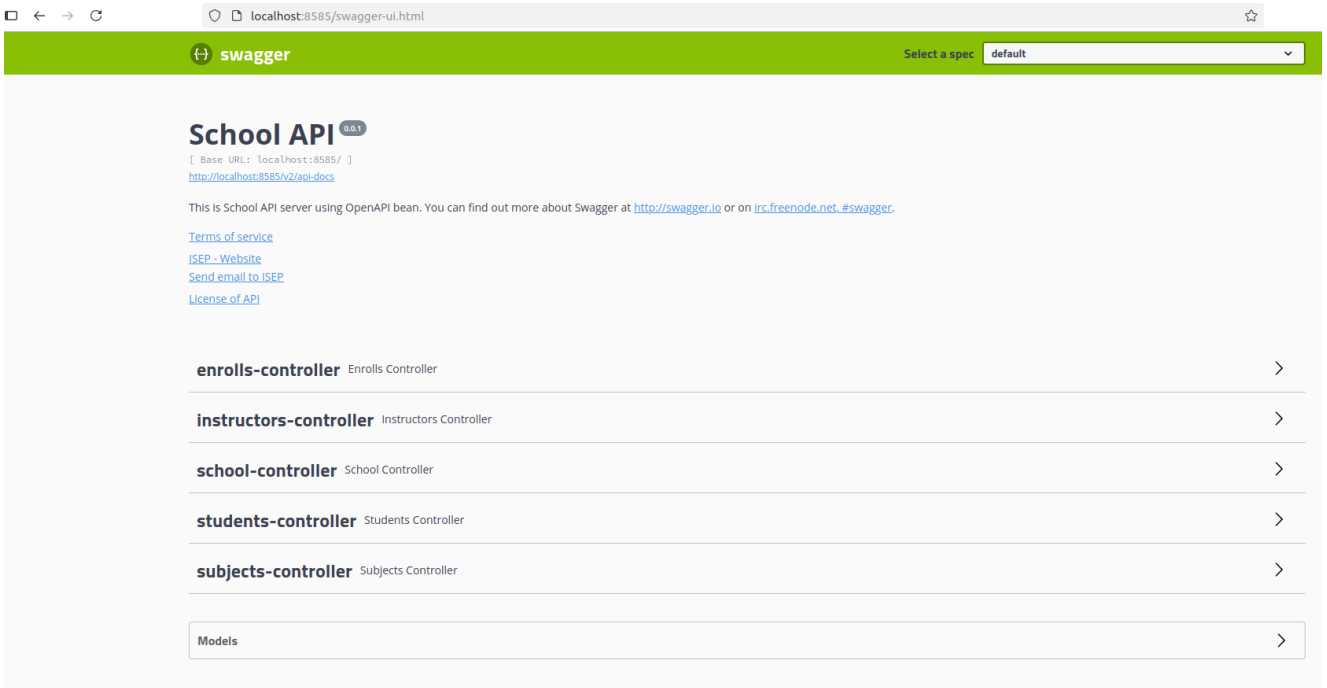
- Install docker desktop in your PC [3].
- In the folder were you extracted `SchoolDiskAPI.zip` run the following command:
  - `docker-compose up -d --build`

```
➜  SchoolDiskAPI docker-compose up -d --build
WARN[0000] /home/pbs/Temp/DSSMV/Old/SchoolDiskAPI/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
Compose can now delegate builds to bake for better performance.
 To do so, set COMPOSE_BAKE=true.
[+] Building 29.6s (12/12) FINISHED
 => [library-api internal] load build definition from Dockerfile
 => => transferring dockerfile: 436B
 => [library-api internal] load metadata for docker.io/library/amazoncorretto:11
 => [library-api internal] load .dockerignore
 => => transferring context: 2B
 => [library-api 1/6] FROM docker.io/library/amazoncorretto:11@sha256:731c45b6f5245537ccd8a764a810af83dc4a5f0cac3349c13d49555b1ae341cc
 => => resolve docker.io/library/amazoncorretto:11@sha256:731c45b6f5245537ccd8a764a810af83dc4a5f0cac3349c13d49555b1ae341cc
 => => sha256:731c45b6f5245537ccd8a764a810af83dc4a5f0cac3349c13d49555b1ae341cc 2.66kB / 2.66kB
 => => sha256:7f8ad20ef5842c8f9f21d92db3bd47eab2070f48299101dcd29fb0ad0cfa8f2d 1.36kB / 1.36kB
 => => sha256:59ba8e66944692dc6b3df745cbc1646e237399b293a9808451ffb60e77e09fc6 2.48kB / 2.48kB
 => => sha256:7234d0a9b6ba1270c7bb653ee62a443a1dbb6fdfa833336edd3c0691c2524d1f 54.63MB / 54.63MB
 => => sha256:0e937dcd3b183f6f63eacb64f6500be3e00b664e51712cee00cf05327cc2c659 153.51MB / 153.51MB
 => => extracting sha256:7234d0a9b6ba1270c7bb653ee62a443a1dbb6fdfa833336edd3c0691c2524d1f
 => => extracting sha256:0e937dcd3b183f6f63eacb64f6500be3e00b664e51712cee00cf05327cc2c659
 => [library-api internal] load build context
 => => transferring context: 27.30MB
 => [library-api 2/6] WORKDIR /opt/app
 => [library-api 3/6] COPY SchoolDiskAPI-0.0.1-SNAPSHOT.jar app.jar
 => [library-api 4/6] RUN echo ##############################
 => [library-api 5/6] RUN echo ##############################
 => [library-api 6/6] RUN sh -c 'touch app.jar'
 => [library-api] exporting to image
 => => exporting layers
 => => writing image sha256:3cf6dccc1165db90e56a7345b3124dbc49401c7bad8b5f97e197f8d48f0afbb8
 => => naming to docker.io/library/java:11
 => [library-api] resolving provenance for metadata file
[+] Running 3/3
 ✔ library-api                  Built
 ✔ Network schooldiskapi_library  Created
 ✔ Container library-api        Started
```

[3] `https://www.docker.com/products/docker-desktop`

Checking and testing

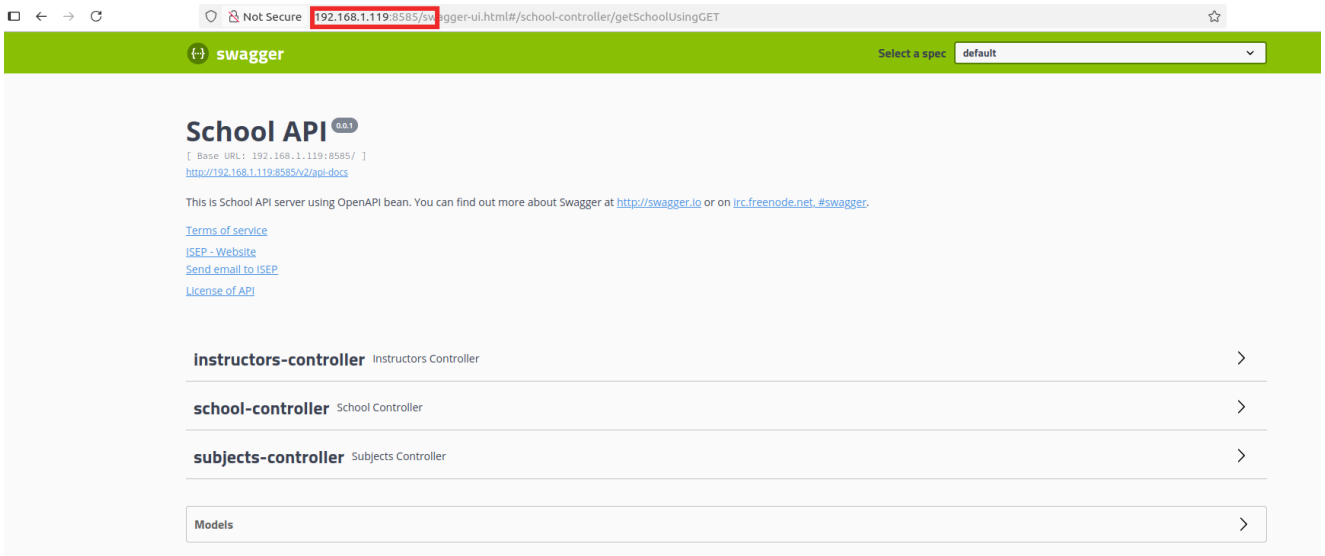
- After boot you have to check if the web service is running:
- Open a browser and enter:
  - `http://localhost:8585/swagger-ui.html`



- Test it using Swagger `https://swagger.io/`

Getting the IP Address

- There are several ways to get the IP address of your PC.
- You can prompt an IA tool with "how to check the ip address of my PC"
- After getting it, check `School Disk API` with the provided IP:



1 Context/Problem

2 School Disk API

3 School Disk Client

4 School Disk Client: Technical Report
- Context/Problem
- Analysis
- Design
- Implementation
- Tests
- Conclusion

Setting up

- To run it on your computer download it from moodle the zip named: `SchoolDiskClient.zip`
- Unzip it.

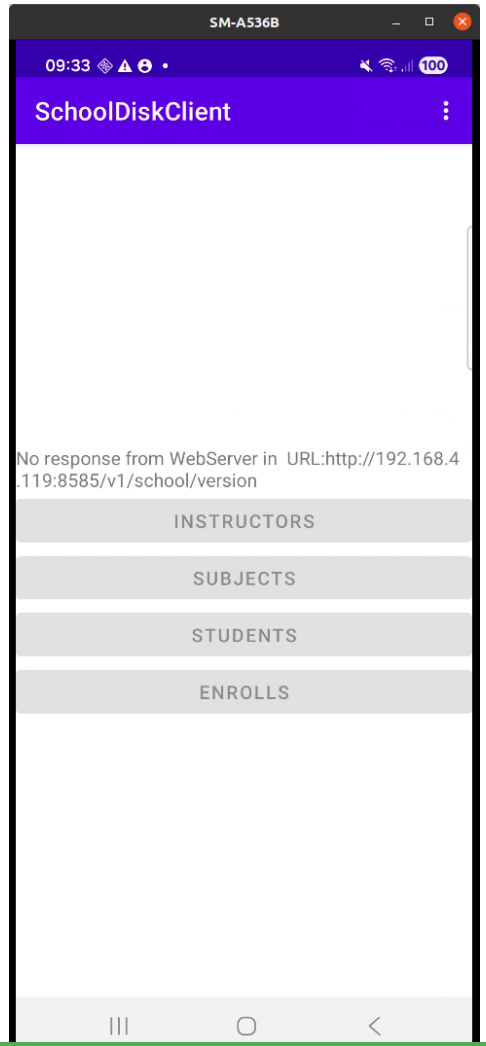
Boot: IntelliJ IDE

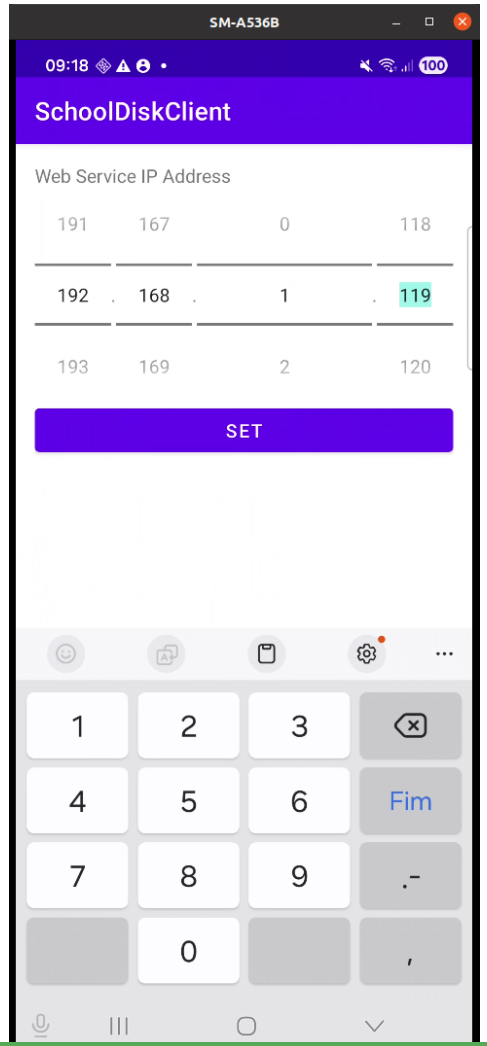
- In order to open and run this project from IntelliJ IDEA it is required the Android plugin

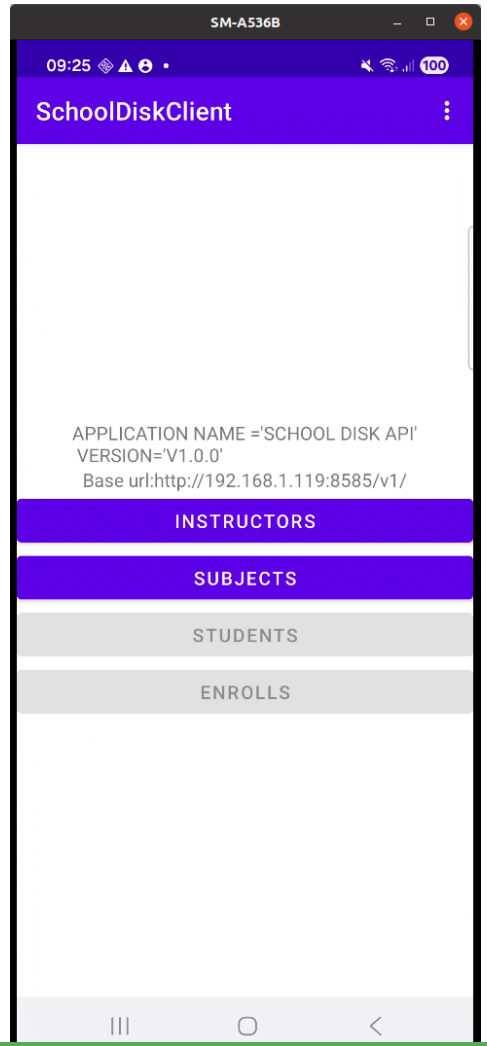


Setting the `School Disk API` IP Address

- Open the Option Menu and select `Settings`, enter `School Disk API` IP Address and click on `Set`







1 Context/Problem

2 School Disk API

3 School Disk Client

4 School Disk Client: Technical Report
- Context/Problem
- Analysis
- Design
- Implementation
- Tests
- Conclusion

1 Context/Problem

2 School Disk API

3 School Disk Client

4 School Disk Client: Technical Report
- Context/Problem
- Analysis
- Design
- Implementation
- Tests
- Conclusion

Context/Problem

- Develop a mobile client app to interact with the `School Disk API` webservice application.
- `School Disk API` provides a set of functionalities to manage a school [4].
  - School features:
    - School has students, instructors and offers subjects.
    - Instructors can teach many subjects.
    - The same subject can be taught by different instructors.
    - Student can attend to many subjects.
  - School functionalities:
    - CRUD Students
    - CRUD Subjects
    - ...

[4] Using `http://localhost:8585/swagger-ui.html`

1 Context/Problem

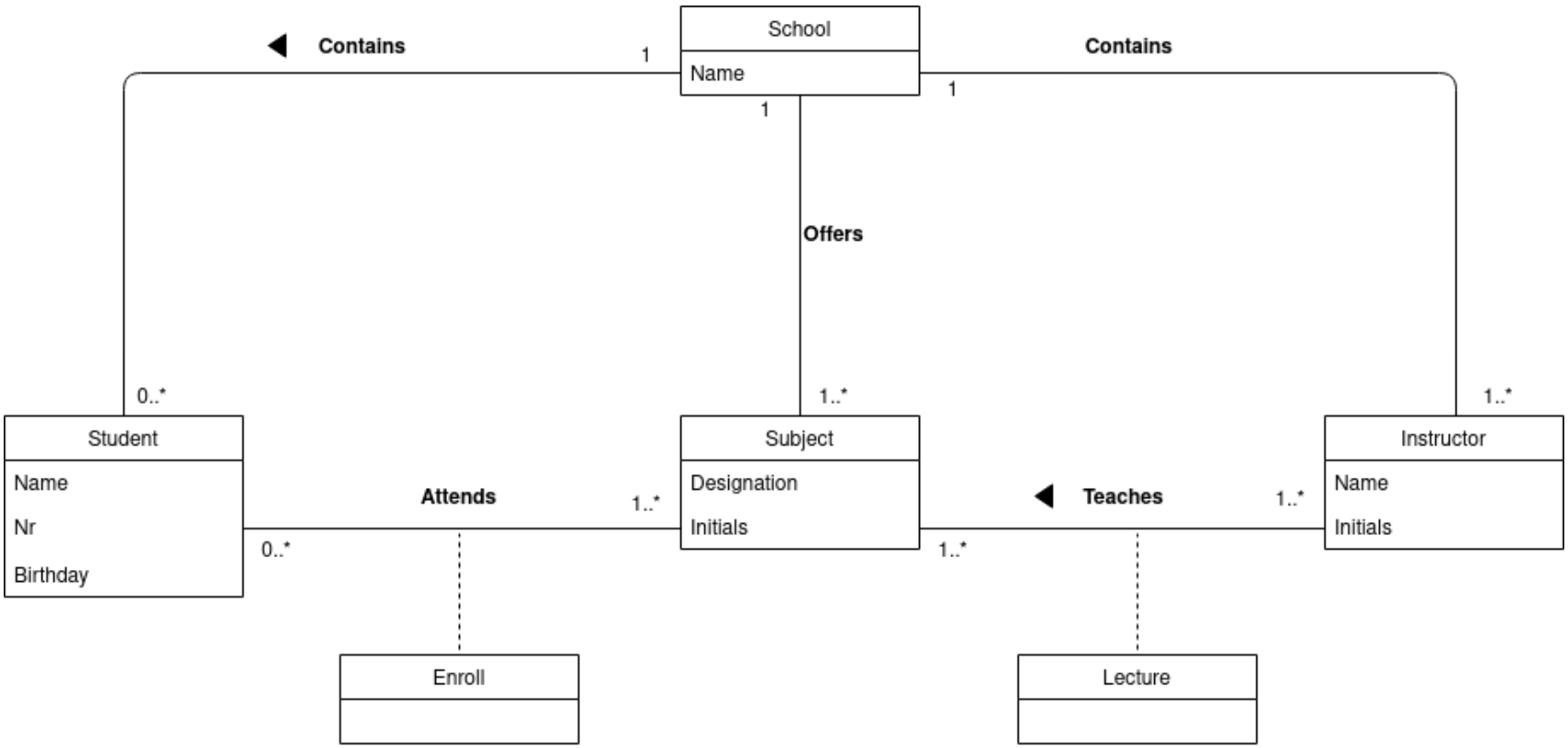
2 School Disk API

3 School Disk Client

4 School Disk Client: Technical Report
- Context/Problem
- Analysis
- Design
- Implementation
- Tests
- Conclusion

Domain Model

- *Explain the purpose of each entity, relationships and cardinality*.



Non-Functional Requirements (I)

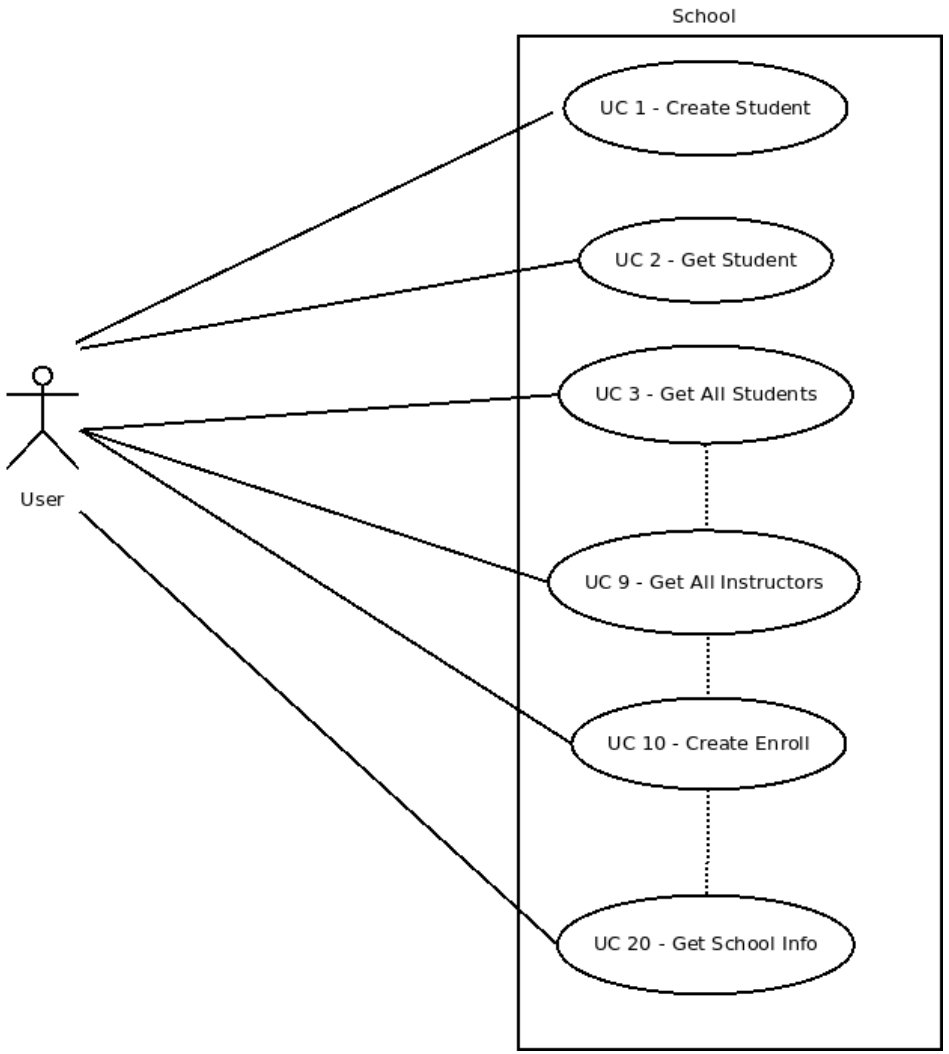
- **Usability**
  - Graphical Interface
- **Reliability**
  - No
- **Performance**
  - No
- **Supportability**
  - Huge amount of tests (unit tests)
  - Android platform
  - Minimal Android API 16

Non-Functional Requirements (II)

- +
  - **Design constraints**
    - No
  - **Implementation constraints**
    - Java language
  - **Interface constraints**
    - Android Graphical Interface

Functional Requirements (I)

- Funcionalities



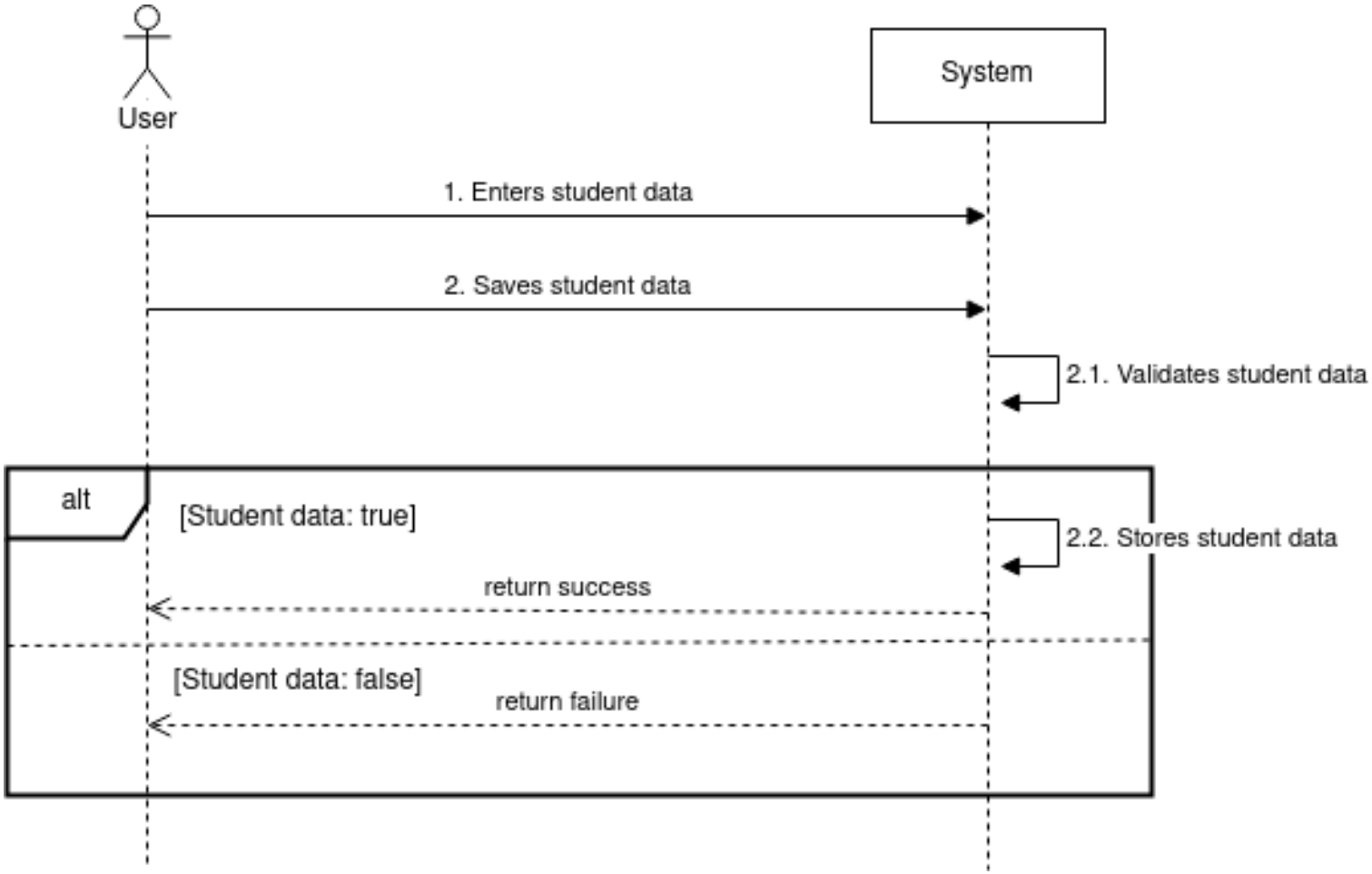
Functional Requirements (II)

- Specification: UC 1 - Create Student

Description	Add one student to the system
Pre-condition	There are no preconditions associated with this use case.
Post-condition	The number of students has to increase one.
Basic path	1. The User enters the student data. 2. The User saves the student data. 3. The System validates the student data. 4. The System stores the student data. 5. The System returns success.
Alternative path	• Invalid data • The System returns failure.

Functional Requirements (III)

- SSD: UC 1 - Create Student



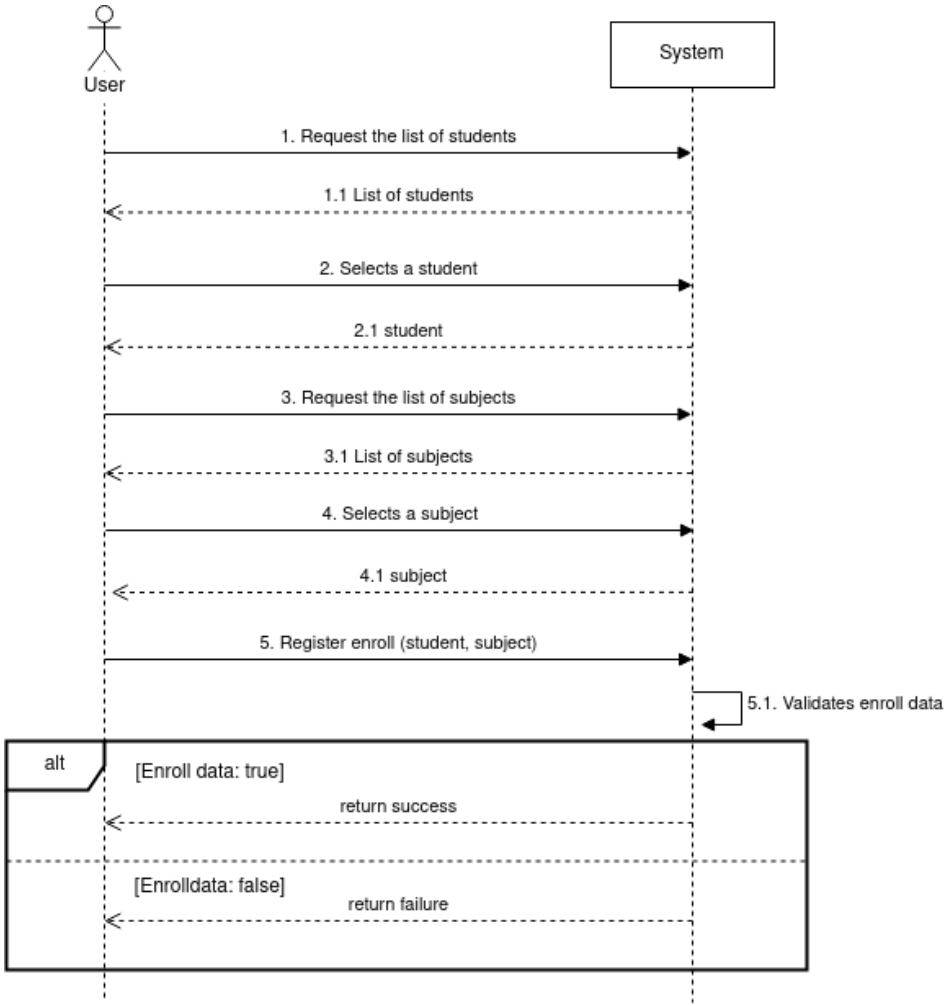
Functional Requirements (IV)

- Specification: UC 10 - Create Enroll

Description	Associate one student to one subject
Pre-condition	The student and the subject must exist in the system
Post-condition	One more Enroll.
Basic path	1. The User requests the list of students. 2. The User selects one student. 3. The User requests the list of subjects. 4. The User selects one subject. 5. The User register the enroll. 6. The System validates the enroll data. 7. The System stores the enroll. 8. The System returns success.
Alternative path	• Invalid data • The System returns failure.

Functional Requirements (V)

- SSD: UC 10 - Create Enroll



1 Context/Problem

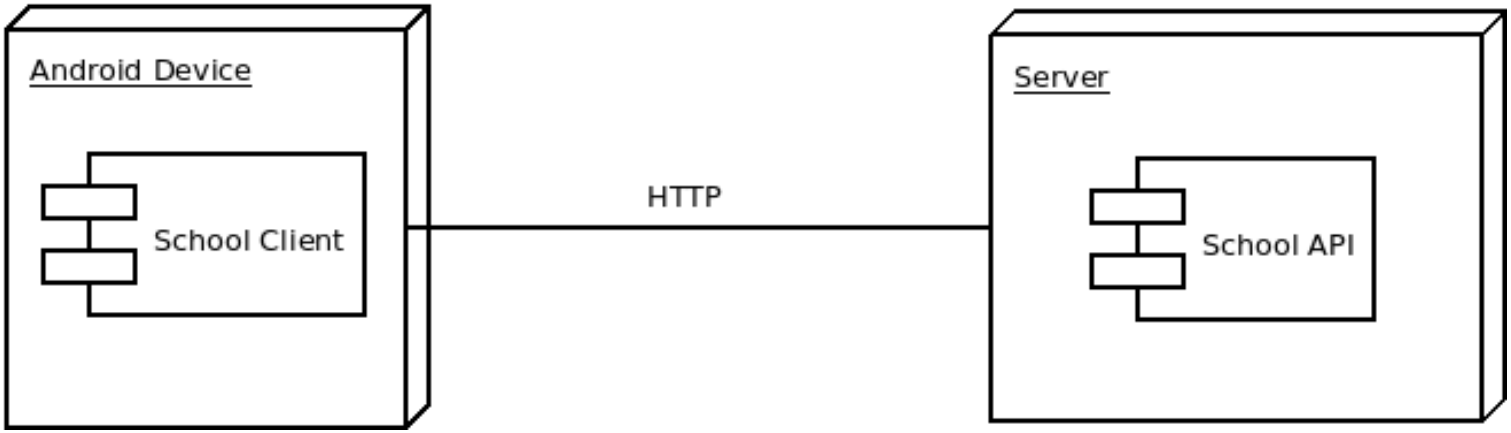
2 School Disk API

3 School Disk Client

4 School Disk Client: Technical Report
- Context/Problem
- Analysis
- Design
- Implementation
- Tests
- Conclusion

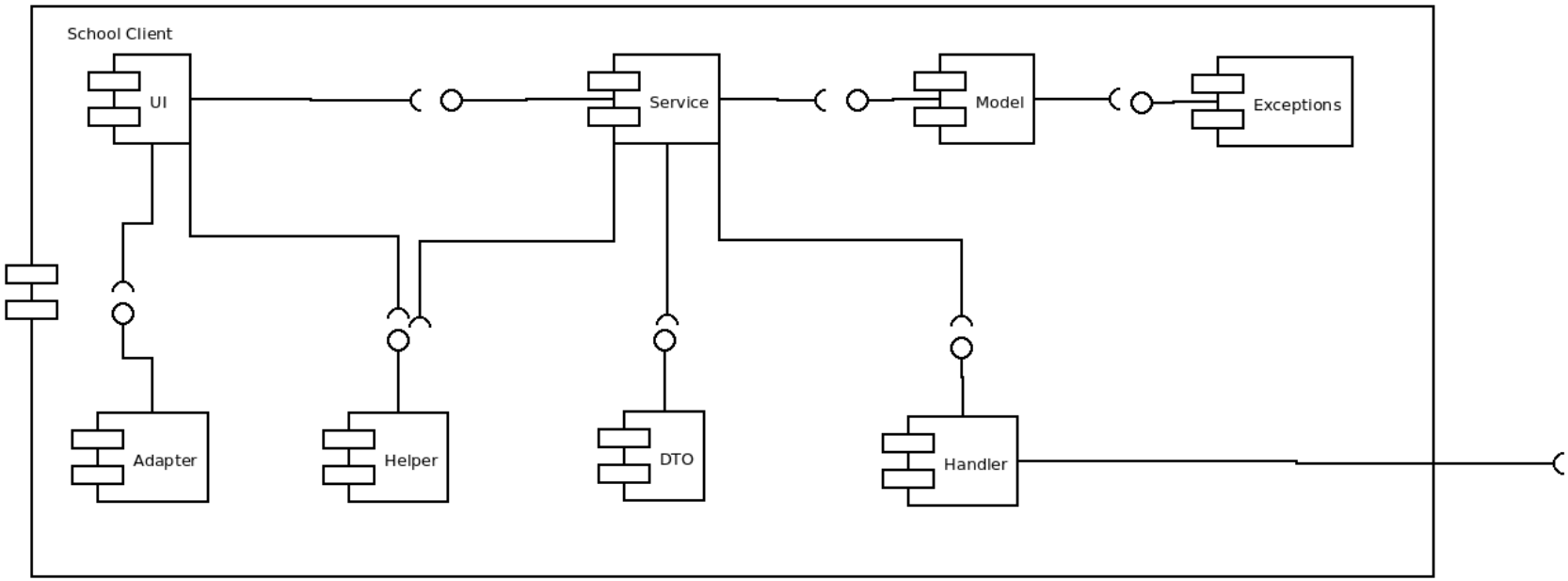
Physical Architecture

- Usage of deployment diagram



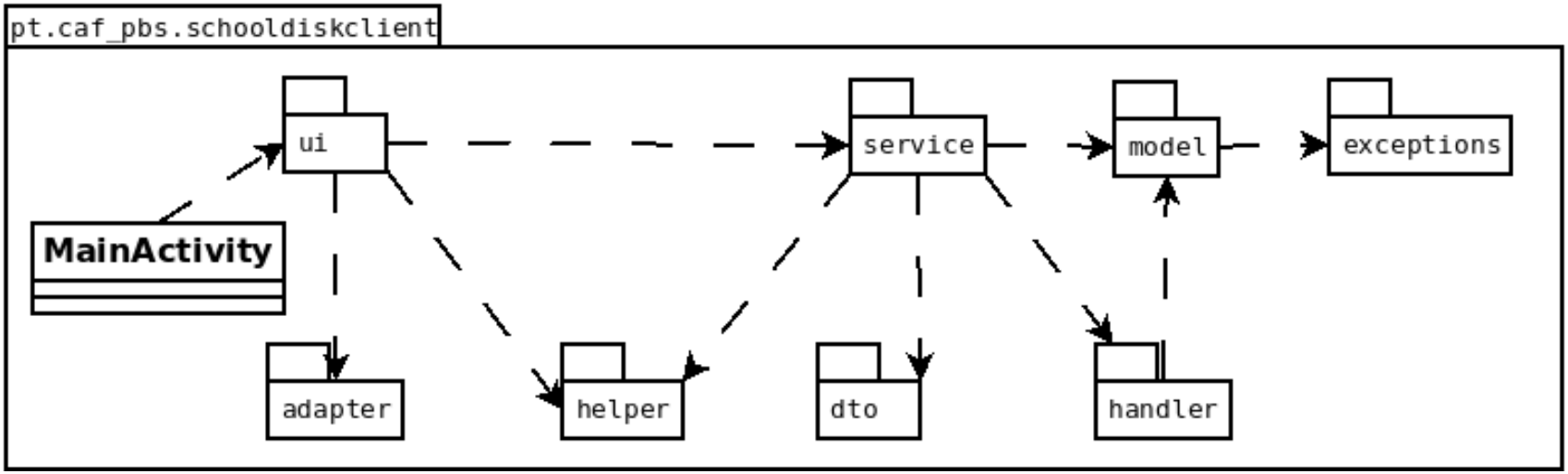
Logical Architecture

- Usage of component diagram



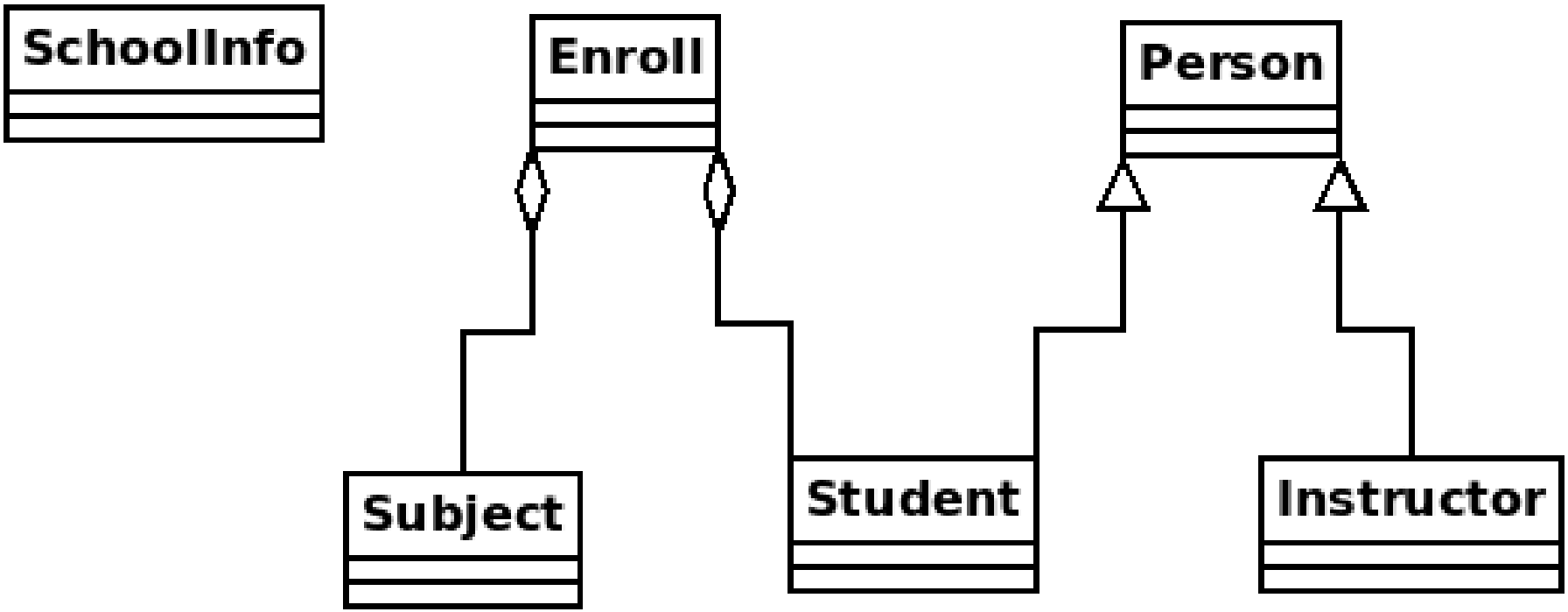
Code Organization

- Usage of package diagram



`model` Class Diagram

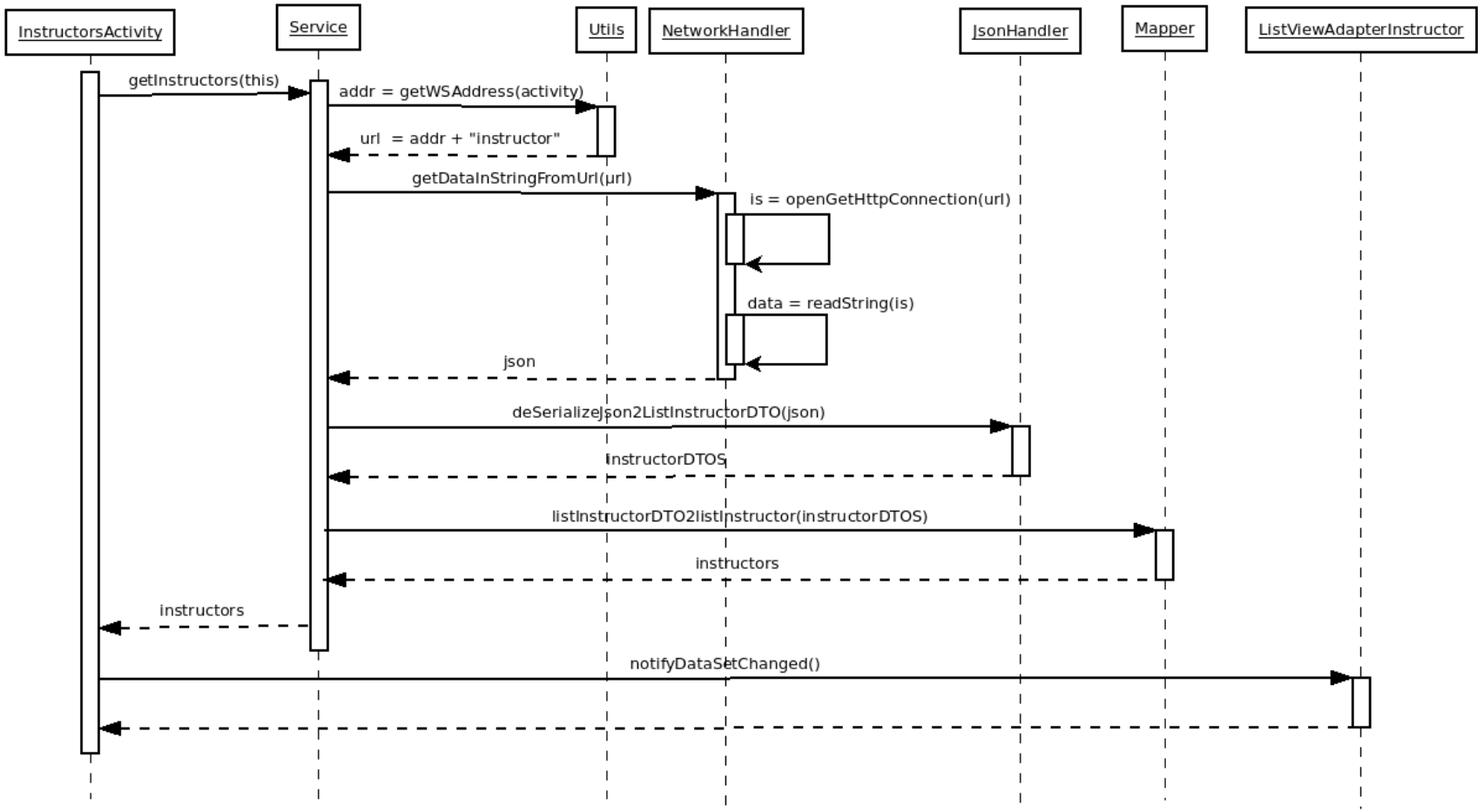
- Usage of class diagram



- The same for the remaining packages

UC 9 - Get All Instructors

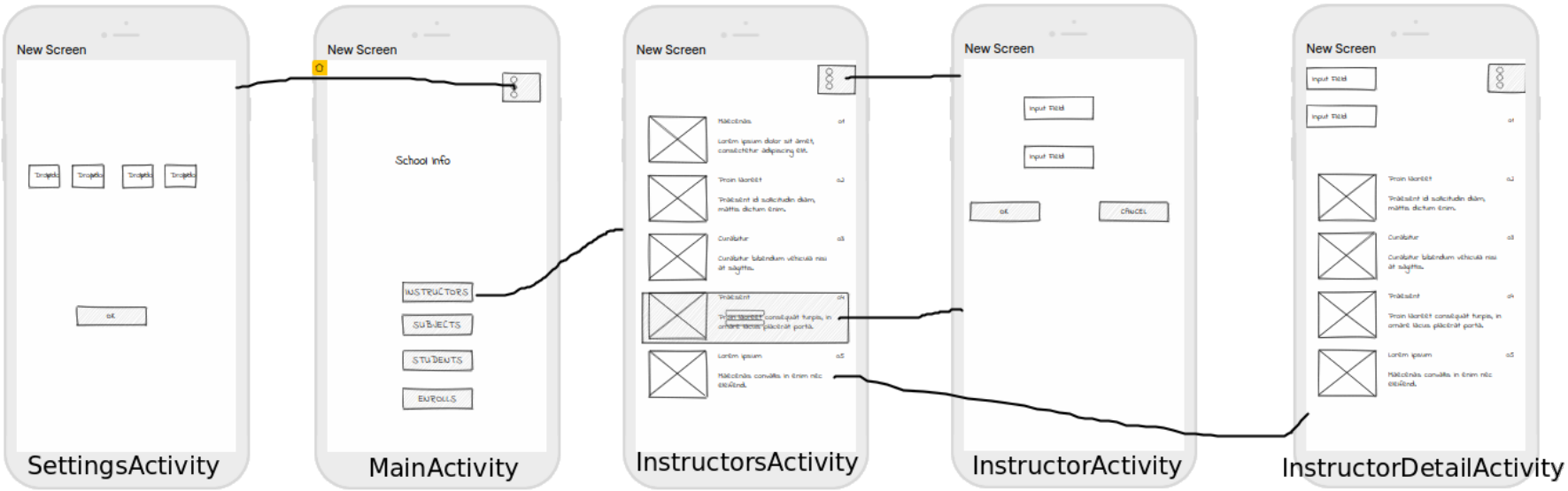
- Usage of sequence diagram



- The same for the remaining UCs

User Interface

- Mock ups



- The same for the remaining screens

1 Context/Problem

2 School Disk API

3 School Disk Client

4 School Disk Client: Technical Report
- Context/Problem
- Analysis
- Design
- Implementation
- Tests
- Conclusion

Overview

- Naming Convention: Camel Case.
- Code snippets cannot be presented as an image. It has to be a listing of text and each line has to be numbered.
  - Use the line number to help the reader
- Explain the implementation of the most important methods or functions (typically, conceptually different):
  - `openGetHttpConnection`
  - `serializeInstructorDTO2Json`
  - `deSerializeJson2ListInstructorDTO`
  - ...

List<InstructorDTO> deSerializeJson2ListInstructorDTO(String resp)

- `deSerializeJson2ListInstructorDTO` method receives a JSON formatted string and return a list of `InstructorDTO` objects (line 1)
- It is supposed that JSON formatted string holds an array.
- ...

```
1  public static List<InstructorDTO> deSerializeJson2ListInstructorDTO(String resp)
       throws JSONException {
2   JSONArray jsonResponse = new JSONArray(resp);
3   List<InstructorDTO> list = new ArrayList<>();
4   for(int i = 0; i<jsonResponse.length();i++){
5     JSONObject jsonChildNode = jsonResponse.getJSONObject(i);
6     String name = jsonChildNode.optString("name");
7     String initials = jsonChildNode.optString("initials");
8     list.add(new InstructorDTO(initials, name));
9   }
10  return list;
11 }
```

1 Context/Problem

2 School Disk API

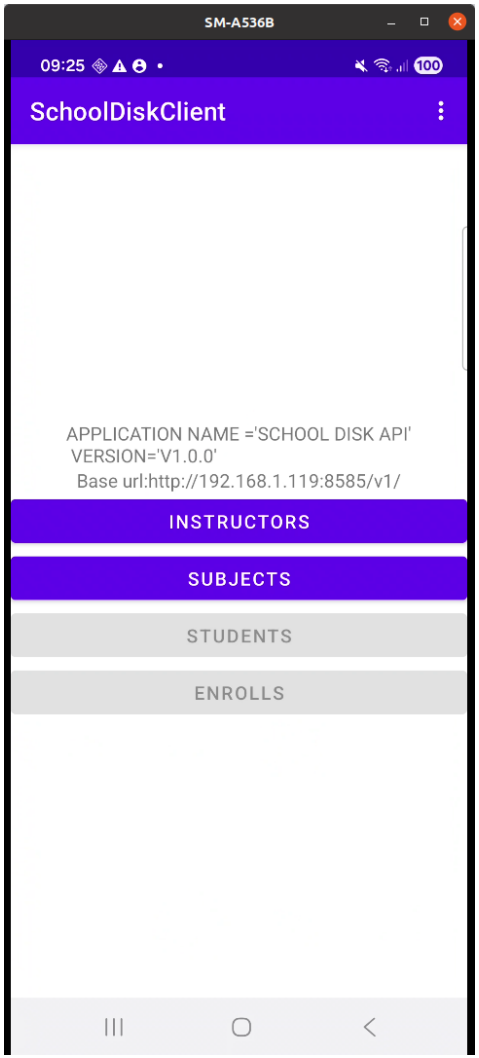
3 School Disk Client

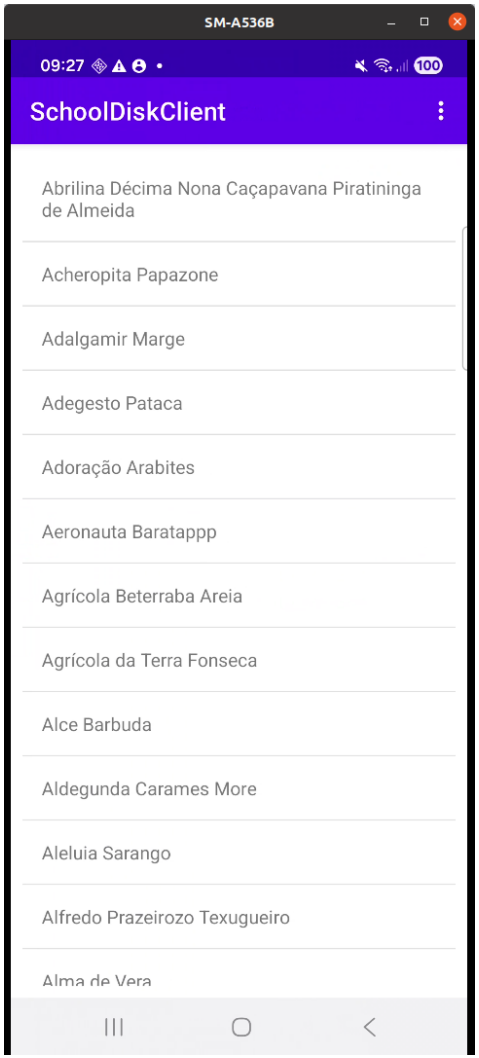
4 School Disk Client: Technical Report
- Context/Problem
- Analysis
- Design
- Implementation
- Tests
- Conclusion

Type of tests

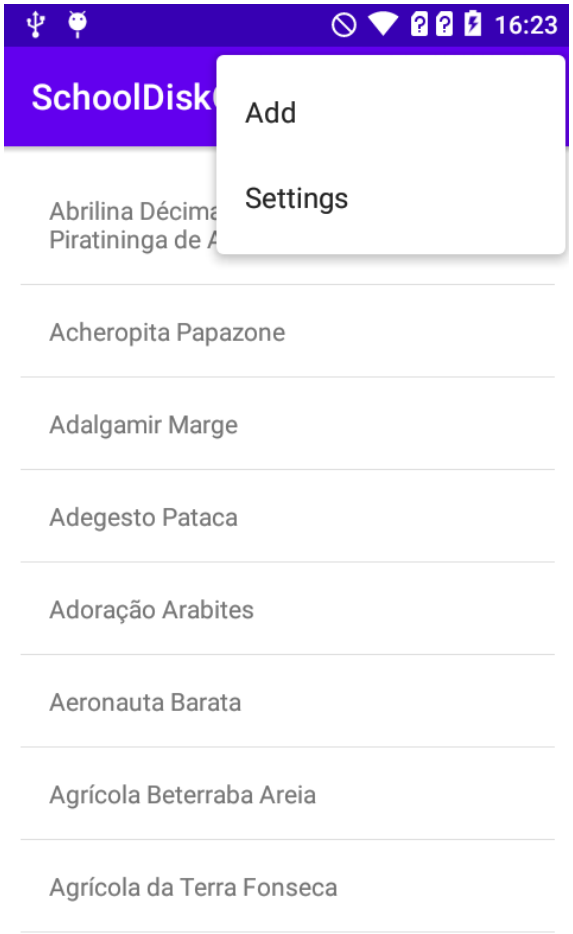
- Unit tests
- Functional tests (end to end tests)
- All tests should be described

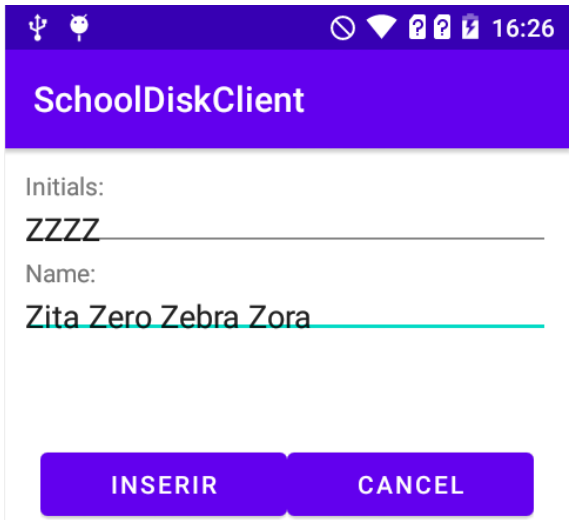
UC 9 - Get All Instructors

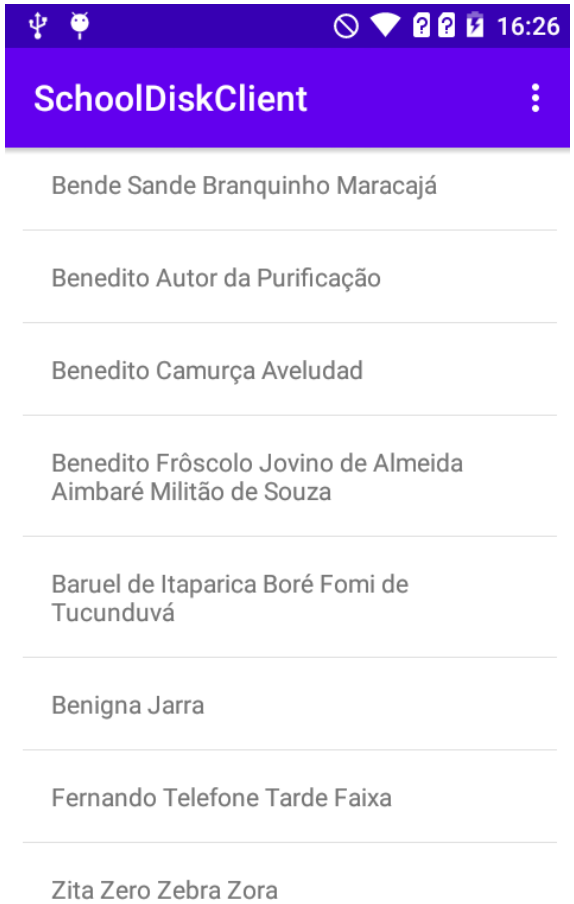




UC 8 - Create an Instructor







1 Context/Problem

2 School Disk API

3 School Disk Client

4 School Disk Client: Technical Report
- Context/Problem
- Analysis
- Design
- Implementation
- Tests
- Conclusion

Conclusion

- Which is working properly and not
- Difficulties